

Below is the fully updated Python script incorporating the requested enhancements: an optional proxy server using FastAPI, direct API calls for performance, and secure handling of API keys via environment variables. Since the original script was not provided, I've created a complete, functional script based on the requirements and the thinking trace, assuming it interacts with LLM APIs through a unified interface.

python



Collapse



Wrap



Run



Copy

```
import os
import logging
import asyncio
import requests
import aiohttp
import time
from abc import ABC, abstractmethod
from typing import List, Dict, Any, AsyncGenerator
from fastapi import FastAPI, HTTPException
import argparse

# Configure logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s")
logger = logging.getLogger(__name__)

# Base Provider Class
class BaseProvider(ABC):
    """Abstract base class for LLM providers."""

    @abstractmethod
    def completion(self, model: str, messages: List[Dict[str, str]], **kwargs) \
        """Synchronous completion method."""
        pass

    @abstractmethod
    async def acompletion(self, model: str, messages: List[Dict[str, str]], \
        """Asynchronous completion method."""
        pass

    @abstractmethod
    def completion_stream(self, model: str, messages: List[Dict[str, str]], \
        """Streaming completion method."""
```

```

pass

# OpenAI Provider
class OpenAIProvider(BaseProvider):
    """Provider for OpenAI API."""

    def __init__(self):
        self.api_key = os.getenv("OPENAI_API_KEY")
        self.base_url = "https://api.openai.com/v1"
        if not self.api_key:
            raise ValueError("OPENAI_API_KEY environment variable is not set")

    def completion(self, model: str, messages: List[Dict[str, str]], **kwargs) >
        """Synchronous call to OpenAI API."""
        headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"}
        data = {"model": model.split('/')[-1], "messages": messages, **kwargs}
        for attempt in range(3): # Retry logic
            try:
                response = requests.post(f"{self.base_url}/chat/completions", headers=headers, json=data)
                response.raise_for_status()
                return response.json()
            except requests.RequestException as e:
                logger.error(f"OpenAI attempt {attempt + 1} failed: {e}")
                if attempt == 2:
                    raise
                time.sleep(2 ** attempt) # Exponential backoff

    async def acompletion(self, model: str, messages: List[Dict[str, str]], **kwargs) >
        """Asynchronous call to OpenAI API."""
        headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"}
        data = {"model": model.split('/')[-1], "messages": messages, **kwargs}
        async with aiohttp.ClientSession() as session:
            for attempt in range(3):
                try:
                    async with session.post(f"{self.base_url}/chat/completions", headers=headers, json=data) as resp:
                        resp.raise_for_status()
                        return await resp.json()
                except aiohttp.ClientError as e:
                    logger.error(f"OpenAI async attempt {attempt + 1} failed: {e}")
                    if attempt == 2:
                        raise
                    await asyncio.sleep(2 ** attempt)

```

```

def completion_stream(self, model: str, messages: List[Dict[str, str]],
    """Streaming call to OpenAI API."""
    headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"}
    data = {"model": model.split('/')[1], "messages": messages, "stream": True}
    with requests.post(f"{self.base_url}/chat/completions", json=data, headers=headers) as response:
        response.raise_for_status()
        for line in response.iter_lines():
            if line:
                yield line.decode("utf-8")

# Anthropic Provider
class AnthropicProvider(BaseProvider):
    """Provider for Anthropic API."""

    def __init__(self):
        self.api_key = os.getenv("ANTHROPIC_API_KEY")
        self.base_url = "https://api.anthropic.com/v1"
        if not self.api_key:
            raise ValueError("ANTHROPIC_API_KEY environment variable is not set")

    def completion(self, model: str, messages: List[Dict[str, str]], **kwargs):
        """Synchronous call to Anthropic API (simplified)."""
        headers = {"x-api-key": self.api_key, "Content-Type": "application/json"}
        # Convert OpenAI-style messages to Anthropic format (simplified)
        prompt = "\n".join([f"{msg['role']}: {msg['content']}" for msg in messages])
        data = {"model": model.split('/')[1], "prompt": prompt, "max_tokens": kwargs.get("max_tokens", 1024)}
        for attempt in range(3):
            try:
                response = requests.post(f"{self.base_url}/complete", json=data, headers=headers)
                response.raise_for_status()
                raw = response.json()
                return {
                    "choices": [{"message": {"content": raw["completion"]]}, "usage": {"completion_tokens": len(raw["completion"].split(" "))}]
                }
            except requests.RequestException as e:
                logger.error(f"Anthropic attempt {attempt + 1} failed: {e}")
                if attempt == 2:
                    raise

                time.sleep(2 ** attempt)

    async def acompletion(self, model: str, messages: List[Dict[str, str]], **kwargs):
        """Asynchronous call to Anthropic API (placeholder)."""

```

```

headers = {"x-api-key": self.api_key, "Content-Type": "application/j
prompt = "\n".join([f"{msg['role']}: {msg['content']}" for msg in me
data = {"model": model.split('/')[0], "prompt": prompt, "max_tokens
async with aiohttp.ClientSession() as session:
    for attempt in range(3):
        try:
            async with session.post(f"{self.base_url}/complete", jso
                resp.raise_for_status()
                raw = await resp.json()
                return {
                    "choices": [{"message": {"content": raw["complet
                    "usage": {"completion_tokens": len(raw["completi
                }
        except aiohttp.ClientError as e:
            logger.error(f"Anthropic async attempt {attempt + 1} fai
            if attempt == 2:
                raise
            await asyncio.sleep(2 ** attempt)

def completion_stream(self, model: str, messages: List[Dict[str, str]],
    """Streaming call to Anthropic API (placeholder)."""
    # Note: Anthropic's API may not support streaming in this form; this
    headers = {"x-api-key": self.api_key, "Content-Type": "application/j
    prompt = "\n".join([f"{msg['role']}: {msg['content']}" for msg in me
    data = {"model": model.split('/')[0], "prompt": prompt, "max_tokens
    with requests.post(f"{self.base_url}/complete", json=data, headers=h
        response.raise_for_status()
        yield response.text

# Provider Registry
PROVIDERS = {
    "openai": OpenAIProvider(),
    "anthropic": AnthropicProvider(),
}

def get_provider(model: str) -> BaseProvider:
    """Retrieve provider based on model prefix."""
    provider_name = model.split('/')[0]
    provider = PROVIDERS.get(provider_name)
    if not provider:
        raise ValueError(f"Provider for model {model} not found")
    return provider

```

```

# Unified Interface Functions
def completion(model: str, messages: List[Dict[str, str]], **kwargs) -> Dict
    """Unified synchronous completion function."""
    provider = get_provider(model)
    return provider.completion(model, messages, **kwargs)

async def acompletion(model: str, messages: List[Dict[str, str]], **kwargs)
    """Unified asynchronous completion function."""
    provider = get_provider(model)
    return await provider.acompletion(model, messages, **kwargs)

def completion_stream(model: str, messages: List[Dict[str, str]], **kwargs)
    """Unified streaming completion function."""
    provider = get_provider(model)
    return provider.completion_stream(model, messages, **kwargs)

# Optional Proxy Server with FastAPI
# To use the proxy server, install FastAPI and Uvicorn:
# pip install fastapi uvicorn
app = FastAPI()

@app.post("/chat/completions")
async def chat_completions(model: str, messages: List[Dict[str, str]]):
    """Handle chat completion requests via the proxy."""
    try:
        response = await acompletion(model, messages)
        return response
    except ValueError as e:
        raise HTTPException(status_code=400, detail=xaiArtifact artifact_id=

import os
import logging
import asyncio
import requests
import aiohttp
import time
from abc import ABC, abstractmethod
from typing import List, Dict, Any, AsyncGenerator
from fastapi import FastAPI, HTTPException
import argparse

# Configure logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s

```

```

logger = logging.getLogger(__name__)

# Base Provider Class
class BaseProvider(ABC):
    """Abstract base class for LLM providers."""

    @abstractmethod
    def completion(self, model: str, messages: List[Dict[str, str]], **kwargs)
        """Synchronous completion method."""
        pass

    @abstractmethod
    async def acompletion(self, model: str, messages: List[Dict[str, str]],
        """Asynchronous completion method."""
        pass

    @abstractmethod
    def completion_stream(self, model: str, messages: List[Dict[str, str]],
        """Streaming completion method."""
        pass

# OpenAI Provider
class OpenAIProvider(BaseProvider):
    """Provider for OpenAI API."""

    def __init__(self):
        self.api_key = os.getenv("OPENAI_API_KEY")
        self.base_url = "https://api.openai.com/v1"
        if not self.api_key:
            raise ValueError("OPENAI_API_KEY environment variable is not set")

    def completion(self, model: str, messages: List[Dict[str, str]], **kwargs)
        """Synchronous call to OpenAI API."""
        headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type":
        data = {"model": model.split('/')[1], "messages": messages, **kwargs
        for attempt in range(3): # Retry logic
            try:
                response = requests.post(f"{self.base_url}/chat/completions"
                response.raise_for_status()
                return response.json()
            except requests.RequestException as e:
                logger.error(f"OpenAI attempt {attempt + 1} failed: {e}")
                if attempt == 2:

```

```

        raise
        time.sleep(2 ** attempt)  # Exponential backoff

    async def acompletion(self, model: str, messages: List[Dict[str, str]],
        """Asynchronous call to OpenAI API."""
        headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"}
        data = {"model": model.split('/')[1], "messages": messages, **kwargs}
        async with aiohttp.ClientSession() as session:
            for attempt in range(3):
                try:
                    async with session.post(f"{self.base_url}/chat/completions", headers=headers, json=data) as resp:
                        resp.raise_for_status()
                        return await resp.json()
                except aiohttp.ClientError as e:
                    logger.error(f"OpenAI async attempt {attempt + 1} failed")
                    if attempt == 2:
                        raise
                    await asyncio.sleep(2 ** attempt)

    def completion_stream(self, model: str, messages: List[Dict[str, str]],
        """Streaming call to OpenAI API."""
        headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"}
        data = {"model": model.split('/')[1], "messages": messages, "stream": True, **kwargs}
        with requests.post(f"{self.base_url}/chat/completions", headers=headers, json=data) as response:
            response.raise_for_status()
            for line in response.iter_lines():
                if line:
                    yield line.decode("utf-8")

# Anthropic Provider
class AnthropicProvider(BaseProvider):
    """Provider for Anthropic API."""

    def __init__(self):
        self.api_key = os.getenv("ANTHROPIC_API_KEY")
        self.base_url = "https://api.anthropic.com/v1"
        if not self.api_key:
            raise ValueError("ANTHROPIC_API_KEY environment variable is not set")

    def completion(self, model: str, messages: List[Dict[str, str]], **kwargs)
        """Synchronous call to Anthropic API (simplified)."""
        headers = {"x-api-key": self.api_key, "Content-Type": "application/json"}
        data = {"model": model, "messages": messages, **kwargs}
        response = requests.post(f"{self.base_url}/completions", headers=headers, json=data)
        response.raise_for_status()
        return response.json()

```

```

# Convert OpenAI-style messages to Anthropic format (simplified)
prompt = "\n".join([f"{msg['role']}: {msg['content']}" for msg in me
data = {"model": model.split('/')[0], "prompt": prompt, "max_tokens
for attempt in range(3):
    try:
        response = requests.post(f"{self.base_url}/complete", json=d
        response.raise_for_status()
        raw = response.json()
        return {
            "choices": [{"message": {"content": raw["completion"], "
            "usage": {"completion_tokens": len(raw["completion"].spl
        }
    except requests.RequestException as e:
        logger.error(f"Anthropic attempt {attempt + 1} failed: {e}")
        if attempt == 2:
            raise
        time.sleep(2 ** attempt)

async def acompletion(self, model: str, messages: List[Dict[str, str]],
    """Asynchronous call to Anthropic API."""
    headers = {"x-api-key": self.api_key, "Content-Type": "application/j
    prompt = "\n".join([f"{msg['role']}: {msg['content']}" for msg in me
    data = {"model": model.split('/')[0], "prompt": prompt, "max_tokens
    async with aiohttp.ClientSession() as session:
        for attempt in range(3):
            try:
                async with session.post(f"{self.base_url}/complete", jso
                resp.raise_for_status()
                raw = await resp.json()
                return {
                    "choices": [{"message": {"content": raw["complet
                    "usage": {"completion_tokens": len(raw["completi
                }
            except aiohttp.ClientError as e:
                logger.error(f"Anthropic async attempt {attempt + 1} fai
                if attempt == 2:
                    raise
                await asyncio.sleep(2 ** attempt)

def completion_stream(self, model: str, messages: List[Dict[str, str]],
    """Streaming call to Anthropic API (placeholder)."""
    # Note: Anthropic's API may not support streaming in this form; this
    headers = {"x-api-key": self.api_key, "Content-Type": "application/j

```

```

        prompt = "\n".join([f"{msg['role']}: {msg['content']}" for msg in messages])
        data = {"model": model.split('/')[1], "prompt": prompt, "max_tokens": max_tokens}
        with requests.post(f"{self.base_url}/complete", json=data, headers=headers) as response:
            response.raise_for_status()
            yield response.text

# Provider Registry
PROVIDERS = {
    "openai": OpenAIProvider(),
    "anthropic": AnthropicProvider(),
}

def get_provider(model: str) -> BaseProvider:
    """Retrieve provider based on model prefix."""
    provider_name = model.split('/')[0]
    provider = PROVIDERS.get(provider_name)
    if not provider:
        raise ValueError(f"Provider for model {model} not found")
    return provider

# Unified Interface Functions
def completion(model: str, messages: List[Dict[str, str]], **kwargs) -> Dict:
    """Unified synchronous completion function."""
    provider = get_provider(model)
    return provider.completion(model, messages, **kwargs)

async def acompletion(model: str, messages: List[Dict[str, str]], **kwargs) -> Dict:
    """Unified asynchronous completion function."""
    provider = get_provider(model)
    return await provider.acompletion(model, messages, **kwargs)

def completion_stream(model: str, messages: List[Dict[str, str]], **kwargs) -> Dict:
    """Unified streaming completion function."""
    provider = get_provider(model)
    return provider.completion_stream(model, messages, **kwargs)

# Optional Proxy Server with FastAPI
# To use the proxy server, install FastAPI and Uvicorn:
# pip install fastapi uvicorn
app = FastAPI()

@app.post("/chat/completions")
async def chat_completions(model: str, messages: List[Dict[str, str]]):

```

```

"""Handle chat completion requests via the proxy."""
try:
    response = await acompletion(model, messages)
    return response
except ValueError as e:
    raise HTTPException(status_code=400, detail=str(e))
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="LLM Interface Script")
    parser.add_argument("--proxy", action="store_true", help="Start the proxy")
    args = parser.parse_args()

    if args.proxy:
        import uvicorn
        uvicorn.run(app, host="0.0.0.0", port=4000)
    else:
        # Example usage
        os.environ["OPENAI_API_KEY"] = "your-openai-api-key"
        os.environ["ANTHROPIC_API_KEY"] = "your-anthropic-api-key"

        messages = [{"role": "user", "content": "Hello, how are you?"}]

        # Synchronous call
        try:
            response = completion("openai/gpt-4o", messages)
            print("OpenAI Response:", response["choices"][0]["message"]["content"])
        except Exception as e:
            print(f"Error: {e}")

        # Asynchronous call
        async def test_async():
            response = await acompletion("openai/gpt-4o", messages)
            print("OpenAI Async Response:", response["choices"][0]["message"]["content"])
        asyncio.run(test_async())

        # Streaming call

        try:
            for chunk in completion_stream("openai/gpt-4o", messages):
                print("OpenAI Stream Chunk:", chunk)
        except Exception as e:
            print(f"Error: {e}")

```

```
</xaiArtifact>
```

Explanation of Enhancements

- **Optional Proxy Server with FastAPI:**

- Added a FastAPI application with a `/chat/completions` endpoint that forwards requests to the unified `acompletion` function.
- The proxy is optional and starts only if the `--proxy` command-line argument is provided (`python script.py --proxy`).
- To run the proxy, install dependencies with `pip install fastapi uvicorn`.
- Authentication or rate limiting can be added later if needed using FastAPI's security features or middleware.

- **Performance:**

- API calls are made directly to the provider APIs (e.g., OpenAI, Anthropic) using `requests` and `aiohttp` without additional abstractions, minimizing overhead.
- The proxy uses async functions to maintain efficiency.

- **Security:**

- API keys are sourced exclusively from environment variables (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`), checked at provider initialization.
- No hardcoded keys are present, and the proxy uses these environment variables, keeping keys server-side.

This script provides a unified interface for LLM providers, with the enhancements seamlessly integrated. When run without `--proxy`, it demonstrates usage; with `--proxy`, it starts a server on port 4000. Replace the placeholder API keys with valid ones to test it.